5.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-5.1 Execute the experiment from Code Fragment 5.1 and compare the results on your system to those we report in Code Fragment 5.2.

R-5.2 In Code Fragment 5.1, we perform an experiment to compare the length of a Python list to its underlying memory usage. Determining the sequence of array sizes requires a manual inspection of the output of that program. Redesign the experiment so that the program outputs only those values of k at which the existing capacity is exhausted. For example, on a system consistent with the results of Code Fragment 5.2, your program should output that the sequence of array capacities are 0, 4, 8, 16, 25, ...

R-5.3 Modify the experiment from Code Fragment 5.1 in order to demonstrate that Python's list class occasionally shrinks the size of its underlying array when elements are popped from a list.

R-5.4 Our DynamicArray class, as given in Code Fragment 5.3, does not support use of negative indices with __getitem__. Update that method to better match the semantics of a Python list.

R-5.5 Redo the justification of Proposition 5.1 assuming that the the cost of growing the array from size k to size $2k$ is $3k$ cyber-dollars. How much should each append operation be charged to make the amortization work?

R-5.6 Our implementation of insert for the DynamicArray class, as given in Code Fragment 5.5, has the following inefficiency. In the case when a resize occurs, the resize operation takes time to copy all the elements from an old array to a new array, and then the subsequent loop in the body of insert shifts many of those elements. Give an improved implementation of the insert method, so that, in the case of a resize, the elements are shifted into their final position during that operation, thereby avoiding the subsequent shifting.

R-5.7 Let A be an array of size $n \geq 2$ containing integers from 1 to $n-1$, inclusive, with exactly one repeated. Describe a fast algorithm for finding the integer in A that is repeated.

R-5.8 Experimentally evaluate the efficiency of the pop method of Python's list class when using varying indices as a parameter, as we did for insert on page 205. Report your results akin to Table 5.5.

R-5.9 Explain the changes that would have to be made to the program of Code Fragment 5.11 so that it could perform the Caesar cipher for messages that are written in an alphabet-based language other than English, such as Greek, Russian, or Hebrew.

R-5.10 The constructor for the CaesarCipher class in Code Fragment 5.11 can be implemented with a two-line body by building the forward and backward strings using a combination of the join method and an appropriate comprehension syntax. Give such an implementation.

R-5.11 Use standard control structures to compute the sum of all numbers in an $n \times n$ data set, represented as a list of lists.

R-5.12 Describe how the built-in sum function can be combined with Python's comprehension syntax to compute the sum of all numbers in an $n \times n$ data set, represented as a list of lists.

Creativity

C-5.13 In the experiment of Code Fragment 5.1, we begin with an empty list. If data were initially constructed with nonempty length, does this affect the sequence of values at which the underlying array is expanded? Perform your own experiments, and comment on any relationship you see between the initial length and the expansion sequence.

C-5.14 The shuffle method, supported by the random module, takes a Python list and rearranges it so that every possible ordering is equally likely. Implement your own version of such a function. You may rely on the randrange(n) function of the random module, which returns a random number between 0 and $n-1$ inclusive.

C-5.15 Consider an implementation of a dynamic array, but instead of copying the elements into an array of double the size (that is, from N to $2N$) when its capacity is reached, we copy the elements into an array with $\lceil N/4 \rceil$ additional cells, going from capacity N to capacity $N + \lceil N/4 \rceil$. Prove that performing a sequence of n append operations still runs in $O(n)$ time in this case.

C-5.16 Implement a pop method for the DynamicArray class, given in Code Fragment 5.3, that removes the last element of the array, and that shrinks the capacity, N, of the array by half any time the number of elements in the array goes below $N/4$.

C-5.17 Prove that when using a dynamic array that grows and shrinks as in the previous exercise, the following series of $2n$ operations takes $O(n)$ time: n append operations on an initially empty array, followed by n pop operations.

C-5.18 Give a formal proof that any sequence of n append or pop operations on an initially empty dynamic array takes $O(n)$ time, if using the strategy described in Exercise C-5.16.

C-5.19 Consider a variant of Exercise C-5.16, in which an array of capacity N is resized to capacity precisely that of the number of elements, any time the number of elements in the array goes strictly below $N/4$. Give a formal proof that any sequence of n append or pop operations on an initially empty dynamic array takes $O(n)$ time.

C-5.20 Consider a variant of Exercise C-5.16, in which an array of capacity N, is resized to capacity precisely that of the number of elements, any time the number of elements in the array goes strictly below $N/2$. Show that there exists a sequence of n operations that requires $\Omega(n^2)$ time to execute.

C-5.21 In Section 5.4.2, we described four different ways to compose a long string: (1) repeated concatenation, (2) appending to a temporary list and then joining, (3) using list comprehension with join, and (4) using generator comprehension with join. Develop an experiment to test the efficiency of all four of these approaches and report your findings.

C-5.22 Develop an experiment to compare the relative efficiency of the extend method of Python's list class versus using repeated calls to append to accomplish the equivalent task.

C-5.23 Based on the discussion of page 207, develop an experiment to compare the efficiency of Python's list comprehension syntax versus the construction of a list by means of repeated calls to append.

C-5.24 Perform experiments to evaluate the efficiency of the remove method of Python's list class, as we did for insert on page 205. Use known values so that all removals occur either at the beginning, middle, or end of the list. Report your results akin to Table 5.5.

C-5.25 The syntax data.remove(value) for Python list data removes only the first occurrence of element value from the list. Give an implementation of a function, with signature remove_all(data, value), that removes *all* occurrences of value from the given list, such that the worst-case running time of the function is $O(n)$ on a list with n elements. Not that it is not efficient enough in general to rely on repeated calls to remove.

C-5.26 Let B be an array of size $n \geq 6$ containing integers from 1 to $n-5$, inclusive, with exactly five repeated. Describe a good algorithm for finding the five integers in B that are repeated.

C-5.27 Given a Python list L of n positive integers, each represented with $k = \lceil \log n \rceil + 1$ bits, describe an $O(n)$-time method for finding a k-bit integer not in L.

C-5.28 Argue why any solution to the previous problem must run in $\Omega(n)$ time.

C-5.29 A useful operation in databases is the ***natural join***. If we view a database as a list of *ordered* pairs of objects, then the natural join of databases A and B is the list of all ordered triples (x,y,z) such that the pair (x,y) is in A and the pair (y,z) is in B. Describe and analyze an efficient algorithm for computing the natural join of a list A of n pairs and a list B of m pairs.

C-5.30 When Bob wants to send Alice a message M on the Internet, he breaks M into n ***data packets***, numbers the packets consecutively, and injects them into the network. When the packets arrive at Alice's computer, they may be out of order, so Alice must assemble the sequence of n packets in order before she can be sure she has the entire message. Describe an efficient scheme for Alice to do this, assuming that she knows the value of n. What is the running time of this algorithm?

C-5.31 Describe a way to use recursion to add all the numbers in an $n \times n$ data set, represented as a list of lists.

Projects

P-5.32 Write a Python function that takes two three-dimensional numeric data sets and adds them componentwise.

P-5.33 Write a Python program for a matrix class that can add and multiply two-dimensional arrays of numbers, assuming the dimensions agree appropriately for the operation.

P-5.34 Write a program that can perform the Caesar cipher for English messages that include both upper- and lowercase characters.

P-5.35 Implement a class, SubstitutionCipher, with a constructor that takes a string with the 26 uppercase letters in an arbitrary order and uses that for the forward mapping for encryption (akin to the self._forward string in our CaesarCipher class of Code Fragment 5.11). You should derive the backward mapping from the forward version.

P-5.36 Redesign the CaesarCipher class as a subclass of the SubstitutionCipher from the previous problem.

P-5.37 Design a RandomCipher class as a subclass of the SubstitutionCipher from Exercise P-5.35, so that each instance of the class relies on a random permutation of letters for its mapping.

Chapter Notes

The fundamental data structures of arrays belong to the folklore of computer science. They were first chronicled in the computer science literature by Knuth in his seminal book on *Fundamental Algorithms* [64].